

RAPPORT COMPLET ET DÉTAILLÉ SUR LE BASH SCRIPTING

1 Introduction au Bash Scripting

Le **bash scripting** consiste à écrire des fichiers texte contenant une suite de commandes interprétées par le shell **Bash (Bourne Again SHell)**.

Il est largement utilisé pour :

- L'automatisation des tâches système
- L'administration serveur
- Le déploiement d'applications
- La surveillance (monitoring)
- Les tâches DevOps (CI/CD, provisioning)

Un script bash est interprété ligne par ligne par `/bin/bash`.

2 Le shebang et les permissions

```
#!/bin/bash
```

👉 Indique quel interpréteur utiliser.

```
chmod +x script.sh  
./script.sh
```

📌 Sans `chmod +x`, le script ne peut pas être exécuté directement.

3 Commandes Bash essentielles (avec explications)

uptime

Affiche depuis combien de temps la machine est allumée et la charge système.

```
uptime
```

Sortie typique :

```
10:12:34 up 5 days, load average: 0.12, 0.08, 0.05
```

read

Permet de lire une entrée utilisateur.

```
read -p "Entrez votre nom : " nom  
echo "Bonjour $nom"
```

df / du

- **df** : espace disque disponible
- **du** : taille des fichiers

```
df -h  
du -sh /var/log
```

grep

Recherche dans un fichier ou une sortie.

```
grep "error" /var/log/syslog  
ps aux | grep nginx
```

netcat (nc)

Outil réseau puissant.

```
nc -zv 192.168.1.10 22
```

➡ Vérifie si le port 22 (SSH) est ouvert.

curl / wget

Téléchargement ou test HTTP.

```
curl -I https://google.com
```

```
wget https://example.com/file.zip
```

4 Variables Bash

Déclaration

```
NAME="Linux"
```

```
echo $NAME
```

Variables système importantes

Variabl e	Description
--------------	-------------

<code>\$0</code>	Nom du script
------------------	---------------

<code>\$1 .. \$9</code>	Arguments
-------------------------	-----------

<code>\${10}</code>	10e argument
---------------------	--------------

<code>\$#</code>	Nombre d'arguments
------------------	-----------------------

<code>\$@</code>	Tous les arguments
------------------	-----------------------

<code>\$?</code>	Code retour
------------------	-------------

<code>\$\$</code>	PID du script
-------------------	---------------

`$RANDO` Nombre aléatoire
`M`

\$RANDOM

```
echo $RANDOM  
echo $((RANDOM % 100))
```

Problème avec `$9.05`

Bash interprète `$9` puis `.05`

 Mauvais :

```
echo $9.05
```

 Correct :

```
echo ${9}.05
```

5 Exporter une variable

```
export APP_ENV=production
```

➔ Disponible pour les scripts enfants.

6 Substitution de commande

Syntaxe recommandée

```
result=$(commande)
```

Exemples puissants

Espace disque utilisé

```
used=$(df -h / | awk 'NR==2 {print $5}')  
echo "Disque utilisé à $used"
```

Processus le plus consommateur

```
top_proc=$(ps aux --sort=-%cpu | head -2)
```

Test réseau

```
if nc -z 8.8.8.8 53; then  
    echo "DNS OK"  
else  
    echo "DNS DOWN"  
fi
```

7 Le code retour \$?

```
cp file /tmp  
if [ $? -eq 0 ]; then  
    echo "Succès"  
else  
    echo "Échec"  
fi
```

👉 Bonne pratique :

```
if cp file /tmp; then  
    echo OK  
else  
    echo FAIL  
fi
```

8 Structures conditionnelles

if / else

```
if [ -f /etc/passwd ]; then
```

```
    echo "Fichier existe"
fi
```

Comparaisons numériques

```
if (( x > 10 )); then
    echo "Grand"
fi
```

case

```
case "$1" in
    start) echo "Start" ;;
    stop) echo "Stop" ;;
    *) echo "Usage" ;;
esac
```

9 Boucles

for

```
for i in {1..5}; do
    echo $i
done
```

while

```
while read line; do
    echo $line
done < fichier.txt
```

ERREUR DANS TON SCRIPT

 Script original :

```
pour i dans { 0. . 10. . 2 }
```

fait

✗ Erreurs :

- pour → for
- fait → done
- Mauvaise syntaxe { 0. . 10. . 2 }

✓ Correction :

```
for i in {0..10..2}; do
  echo "Bienvenue $i fois"
done
```

10 Fichiers Bash importants

Fichier	Rôle
/etc/profile	Configuration globale
e	login
~/.bash_profile	Config utilisateur login
file	
~/.bashrc	Shell interactif
/etc/bashrc	Global interactif

📌 Bonne pratique :

```
source ~/.bashrc
```

11 Cron (planification)

```
crontab -e
```

Format :

```
* * * * * commande
```

Exemples :

```
0 2 * * * /backup.sh  
*/5 * * * * /check_disk.sh
```

12 Script de monitoring (plugin Nagios)

```
#!/bin/bash  
used=$(df / | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
```



```
if (( used > 90 )); then  
    echo "CRITICAL"  
    exit 2  
elif (( used > 80 )); then  
    echo "WARNING"  
    exit 1  
else  
    echo "OK"  
    exit 0  
fi
```

13 SSH et sécurité

/etc/ssh/sshd_config

- PermitRootLogin no
- PasswordAuthentication no
- PubkeyAuthentication yes

- Port 2222
-

14 Authentification par clé SSH

```
ssh-keygen -t rsa -b 4096
ssh-copy-id user@host
ssh user@host
```

- 📌 Clé privée = secrète
 - 📌 Clé publique = serveur
-

15 SCP avec clé

```
scp -i ~/.ssh/id_rsa fichier root@host:/root/
```

16 Connexions multiples (boucle SSH)

```
while read host; do
    ssh devops@$host uptime
done < remhosts
```

17 Analyse de `multios_websetup.sh`

- ✓ Bonne détection OS
- ✗ Message Ubuntu incorrect
- ✗ Pas de `set -euo pipefail`
- ✗ `/tmp` non sécurisé
- ✓ Bonne automatisation multi-OS

🔧 Améliorations recommandées :

- `mktemp`

- `trap cleanup EXIT`
 - Vérifier téléchargements
 - Logs
 - Sécurité sudo
-

18 Keylogger (IMPORTANT)

Je **ne peux pas** fournir de code ou tutoriel pour créer un keylogger.
C'est **illégal et malveillant**.

Alternatives légales

- `auditd`
 - `journalctl`
 - `sudo logs`
 - `acct / lastcomm`
 - SIEM (ELK, Wazuh)
-

19 Debug Bash

```
bash -n script.sh    # vérification syntaxe
bash -x script.sh    # debug ligne par ligne
set -euo pipefail    # robustesse
```



Conclusion

Le bash scripting est un **pilier fondamental de Linux, DevOps et Cloud**.

Bien maîtrisé, il permet :

- ✓ Automatisation
- ✓ Sécurité
- ✓ Monitoring
- ✓ Déploiement
- ✓ Administration serveur